

Phase Performance Optimization P1

Implementation report · Task mutation local patch + background reconciliation · Conservative P1 only (no optimistic UI, no Transaction Engine migration)

Status:

P1 COMPLETE

Successful mutations patch the returned task immediately. Forecast and gates reconcile in the background without blocking the House Details UI.

1. Files changed

File	Role
lib/homes/patch-home-task.ts	patchHomeTask, mergeHomeTask, applyForecastReconcileToHome
lib/homes/mutation-reconcile.ts	Kind policy (gates / N/A skip-recompute), debounce constant
lib/homes/mutation-perf.ts	Dev-only performance.mark / measure (T0-T5)
lib/homes/patch-home-task.test.ts	Unit tests (7 passed)
components/homes/home-detail-page.tsx	Patch + coalesced reconcile wiring
components/task-modal.tsx	Passes { task, kind } on all mutation paths

2. Previous mutation flow

Mutation

- wait for API
- GET /forecast
- GET /gates
- setHome(full home)
- entire House Details page rerenders

The superintendent waited for expensive forecast reconciliation before the confirmed task state was reliably reflected (or saw a second full-home replace after an initial patch on inline complete).

3. New mutation flow

Mutation

- server confirms
- immediately merge returned task into local home.tasks
- UI reflects new task state
- debounced background reconcile (forecast fields only; gates only when needed)

Forecast correctness is preserved; timing of reconciliation moved off the visual critical path.

4. How local task patching works

`patchHomeTask(home, updated)` finds the task by id, merges server fields onto that row, and preserves sibling tasks and unrelated home data. Updates use functional `setHome` / selected-task merges — no direct mutation of state objects.

Applied consistently after successful responses for Complete (inline + modal), Start (modal), Mark N/A (card + modal), Schedule, Reschedule, and related modal paths (cancel-schedule, confirm, mark-applicable).

5. How forecast reconciliation works

- Does **not** await forecast before showing the mutation result.
- After patch, schedules a background reconcile for the home.
- Default path: `GET /api/homes/[id]/forecast` (recompute + persist as before).
- N/A path: `GET /api/homes/[id]` — uses already-persisted forecast from `POST /not-applicable`.
- Applies via `applyForecastReconcileToHome`: home-level forecast fields + per-task forecast/punch-derived fields only.

6. How requests are coalesced

- Debounce: **350ms** (`FORECAST_RECONCILE_DEBOUNCE_MS`).
- Rapid mutations on the same home share one pending reconcile.
- `refreshGates` ORs across coalesced kinds.
- `skipRecompute` stays true only if *all* coalesced kinds allow it (N/A-only batches).
- Only the latest in-flight generation's result is applied.

7. How stale-response protection works

Background responses never call full `setHome(remote)` for mutation follow-up. They merge forecast-derived fields onto the current local home. Local status, scheduled dates, confirmation fields, and N/A fields from newer mutations are preserved even if an older forecast payload still shows prior status.

Covered by unit test: complete/start local status survives a stale forecast that still lists Confirmed.

8. Which mutations still refresh gates

Mutation	Gates refresh?	Why
Complete / Start / N/A / Schedule / Reschedule	No	Punch-based gate statuses are unchanged by these actions

Mutation	Gates refresh?	Why
Punch list updates	Yes	Open punch counts can change critical-gate blocking
Catch-up / generate schedule	Full refreshHomeData	Explicit bulk refresh path (unchanged)

Policy is documented in `lib/homes/mutation-reconcile.ts`. Server category/punch gate checks on schedule/reschedule are unchanged.

9. How N/A duplicate forecast was eliminated

`POST /not-applicable` already runs `computeHomeForecastAndPersist`. The client no longer immediately calls `GET /forecast` (which would recompute again). It patches the returned task, then background-loads `GET /homes/[id]` to pick up persisted forecast fields.

10. Performance instrumentation added

Development-only (no-op in production builds):

- **T0** — action initiated
- **T1** — mutation request sent (inline complete)
- **T2** — mutation response received
- **T3** — local task patch applied
- **T4** — background reconciliation started
- **T5** — background reconciliation completed

Uses `performance.mark / performance.measure`. No analytics vendor. No PII. No production console spam.

11. Tests performed

- `lib/homes/patch-home-task.test.ts` — **7 passed**
- Patch merges only the matching task; siblings preserved
- Stale forecast cannot revert newer local status
- Gate refresh skipped for the five high-frequency kinds
- N/A marked as forecast-already-persisted

Manual field scenarios (rapid complete, A then B before reconcile, mobile viewport) should be verified in the running app; no Transaction Engine or API contract tests were required for P1.

12. Remaining performance bottlenecks

- Still wait for mutation API before showing success (no true optimistic UI — P2).
- Full House Details React tree still rerenders on `setHome` (memoization / split state — later phase).

- Schedule/reschedule server guard query cost unchanged.
- Flow Start still refetches `/api/flow` (Flow list page, not House Details).

Approximate timing (expected)

Interval	Expected
Mutation response → visible task update	~0–16ms after JSON parse (local patch)
Background reconciliation	350ms debounce + forecast/home RTT (often 500ms–2s), non-blocking

Success criteria checklist

- Successful mutation updates the affected task without waiting for full forecast reconciliation
- Single task mutation does not unnecessarily replace/reload the entire home for status display
- Forecast correctness preserved (background reconcile)
- Gate correctness preserved (refresh when punch state can change)
- Rapid mutations protected against stale background overwrites
- Mark N/A no longer double-computes forecast
- Scheduling / dependency / confirmation semantics unchanged
- No Transaction Engine migration
- No true optimistic UI (P2 deferred)

Explicitly not in this phase: optimistic mutations, rollback, offline queue, Transaction Engine status types, API redesign, React Query / SWR / Redux / Zustand.